

Eksempelbeamer

Basic reinforcement-learning concepts and applications in physics

FYS4411 — Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

Week 1

Learning goals for Week 1

By the end of the lecture, you should be able to:

- explain why reinforcement learning differs structurally from fixed-data learning;
- identify the agent, environment, state, action, reward, and policy in a new problem;
- describe the environment through a transition–reward probability kernel;
- state the Markov property and explain why the state representation matters;
- define trajectories, returns, policy objectives, and value functions;
- formulate simple problems from quantum technology, molecular dynamics, and robotics as RL problems.

1. Why reinforcement learning and agentic AI?
2. From neural networks to policies
3. The agent–environment interaction
4. States, actions, rewards, transitions, and policies
5. Trajectories, returns, objectives, and values
6. Three application pathways in physics
7. Where the course goes next

Why FYS4411?

Many scientific problems are not merely about predicting a quantity. They require a sequence of decisions that changes the future experiment or simulation.

Examples include:

- choosing control pulses for a quantum system;
- deciding where to sample a molecular configuration space;
- stabilizing a robot under disturbances;
- selecting tools, experiments, or computations in an agentic workflow.

Takeaway. The common structure is *interaction*: decisions influence which data and opportunities appear next.

Reinforcement learning in one sentence

Working definition

Reinforcement learning studies how an **agent** can learn a **policy** for choosing actions through interaction with an **environment**, so as to obtain large accumulated reward.

The basic loop is

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

The challenge is not only to select a good action now, but to account for how that action changes future states and future rewards.

Ordinary optimization versus sequential decisions

One-shot optimization

$$\theta^* \in \arg \min_{\theta} L(\theta).$$

Choose parameters once,
then evaluate the result.

Sequential decision-making

$$A_0, A_1, A_2, \dots$$

Each action changes the
state in which the next
action will be chosen.

A locally attractive action may be globally poor if it destroys useful future possibilities.

Two reasons RL is different

1. **Actions generate future data.**

$$S_t \xrightarrow{A_t} S_{t+1}.$$

The distribution of later observations depends on the policy itself.

2. **Consequences may be delayed.** A useful action may receive little immediate reward, while enabling a large reward much later.

Therefore, the quality of an action must be judged in the context of a trajectory, not only by its immediate effect.

Three recurring challenges in physics

Quantum technology

Uncertainty

Measurements are probabilistic, may disturb the system, and often reveal only partial information.

Molecular dynamics

Complexity

Configuration spaces are enormous and relevant transitions may be rare.

Robotics and control

Real-time dynamics

Actions must continuously respond to a changing, disturbed physical system.

RL provides one language for all three: states, actions, feedback, trajectories, and long-term objectives.

A few motivating applications

- **Quantum control:** learn pulse sequences for state preparation, gates, sensing, or error correction.
- **Molecular simulation:** guide enhanced sampling, rare-event discovery, reaction-path search, or adaptive simulation.
- **Robotics:** learn locomotion, manipulation, navigation, and disturbance rejection.
- **Games and planning:** learn strategies when reward is delayed and the search space is large.
- **Agentic AI:** choose tools, gather information, and adapt a multi-step plan from observations.

Takeaway. The same formal framework can describe very different physical and computational systems.

Connection to agentic AI

An agentic system repeatedly observes context, chooses an action, and receives a new observation:

$$\text{context}_t \rightarrow \text{tool call or decision}_t \rightarrow \text{observation}_{t+1}.$$

Possible actions include:

- querying a database or running code;
- asking for missing information;
- selecting the next subtask;
- stopping and returning an answer.

Important nuance: not every agentic system is trained with RL, but RL supplies a precise language for sequential actions, feedback, and long-term objectives.

What RL is not

- It is not simply “a neural network with rewards.” Neural networks are optional function approximators.
- It is not identical to optimal control. RL is especially useful when dynamics, objectives, or gradients are unavailable or only partly known.
- It is not always model-free. An RL agent may use a known or learned physical model.
- It does not remove the need for physics. State design, constraints, symmetries, and reward design often determine whether learning is meaningful.

Neural networks as parametrized functions

A feed-forward neural network is a parametrized map

$$f_{\theta} : X \rightarrow Y,$$

where θ denotes its trainable weights and biases.

A schematic multilayer network has the form

$$f_{\theta}(x) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1 x + b_1) \cdots) + b_{L-1}) + b_L.$$

The precise architecture is not the main issue today. The key point is that training means selecting θ so that the function behaves well according to an objective.

The fixed-data setting

In supervised learning, one is commonly given a dataset

$$D = \{(x_i, y_i)\}_{i=1}^N.$$

A loss function ℓ measures the mismatch between prediction and target, giving the empirical objective

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i), \quad \theta^* \in \arg \min_{\theta \in \Theta} L(\theta).$$

The data exist before optimization begins.

The structural assumption behind fixed data

Changing θ changes the predictions $f_\theta(x_i)$ and therefore the loss. It does **not** change:

- the inputs x_i ;
- the targets y_i ;
- the order in which examples appear;
- the data-generating distribution represented by the dataset.

Takeaway. The model is trained on data that already exist; it does not act on the data-generating process.

When outputs become actions

Reinforcement learning begins when the model output is used to act:

$$\pi_{\theta} : S \rightarrow A.$$

In the deterministic case,

$$A_t = \pi_{\theta}(S_t).$$

The output is no longer merely a prediction to compare with a label. It is an intervention that changes the future evolution of the environment.

$$S_t \xrightarrow{\pi_{\theta}} A_t \longrightarrow S_{t+1}.$$

Deterministic and stochastic policies

Deterministic policy

$$\pi : S \rightarrow A, \quad A_t = \pi(S_t).$$

The same state always gives the same action.

Stochastic policy

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

For each state, the policy defines a probability distribution over actions.

Stochastic policies can represent exploration, randomized control, and genuine uncertainty in action selection.

A neural network can parametrize a policy

For a finite action set, a network may output a score $z_\theta(s, a)$ for each action. A softmax layer converts the scores into probabilities:

$$\pi_\theta(a \mid s) = \frac{\exp(z_\theta(s, a))}{\sum_{b \in A(s)} \exp(z_\theta(s, b))}.$$

This guarantees

$$\pi_\theta(a \mid s) \geq 0, \quad \sum_{a \in A(s)} \pi_\theta(a \mid s) = 1.$$

The network is now inside a feedback loop rather than only inside a prediction pipeline.

Fixed data versus an interactive feedback loop

Fixed-data learning

$$x_i \longrightarrow f_{\theta}(x_i) \longrightarrow \ell(f_{\theta}(x_i), y_i).$$

The example is fixed independently of the current model.

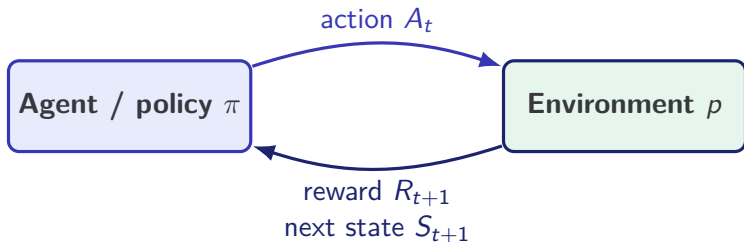
Reinforcement learning

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

The action helps determine the next input to the policy.

This feedback structure is the central conceptual shift from ordinary function fitting to sequential decision-making.

The agent–environment loop



At each time t , the policy uses the current state S_t to choose A_t .
The environment then produces the reward and next state.

The convention

$$(S_t, A_t) \mapsto (R_{t+1}, S_{t+1})$$

emphasizes the causal order:

1. the state S_t is available;
2. the policy chooses A_t ;
3. the action is applied;
4. the environment returns R_{t+1} and S_{t+1} .

The reward index is $t + 1$ because it is observed *after* the action at time t .

Trajectories and episodes

Repeated interaction generates a trajectory

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

A finite realization can be written

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, a_{T-1}, r_T, s_T).$$

- A **trajectory** is a sequence generated by the interaction.
- An **episode** is a trajectory that terminates after a finite number of steps.
- A continuing task may have no natural terminal time.

State, action, and reward spaces

We introduce three sets:

$$S, \quad A, \quad R \subseteq \mathbb{R},$$

with random variables

$$S_t \in S, \quad A_t \in A, \quad R_{t+1} \in R.$$

State S_t : information available when the action is chosen;

Action A_t : the agent's decision or intervention;

Reward R_{t+1} : numerical feedback produced after the action.

Available actions and admissibility

Not every action must be possible in every state. Let

$$A(s) \subseteq A$$

be the nonempty set of actions available in state s .

A pair (s, a) is **admissible** when $a \in A(s)$. The set of admissible state–action pairs is

$$D = \{(s, a) : s \in S, a \in A(s)\} \subseteq S \times A.$$

Physical constraints often enter through $A(s)$: actuator limits, forbidden gates, energy bounds, collision constraints, or experimental safety rules.

The transition–reward probability kernel

The environment response is described by

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

For each admissible $(s, a) \in D$, the kernel gives a probability distribution over possible pairs $(s', r) \in S \times R$.

Takeaway. The policy describes how the agent chooses actions; the kernel describes how the environment responds.

Normalization, transitions, and one-step rewards

For each admissible (s, a) ,

$$p(s', r \mid s, a) \geq 0, \quad \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) = 1.$$

The marginal transition probability is

$$p(s' \mid s, a) = \sum_{r \in R} p(s', r \mid s, a),$$

and the expected one-step reward is

$$r(s, a) = \sum_{s' \in S} \sum_{\rho \in R} \rho p(s', \rho \mid s, a).$$

The Markov property

Let the history up to time t be

$$H_t = (S_0, A_0, R_1, \dots, S_t, A_t).$$

The Markov property states that

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid H_t) = p(s', r \mid S_t, A_t).$$

Once the current state and action are known, the earlier history provides no additional information for predicting the one-step future.

What should the state contain?

The Markov assumption is not merely a property of the physical system. It also depends on how we define the state.

A useful state should summarize the information needed to predict the next transition, such as:

- position and velocity rather than position alone;
- a quantum-state estimate or measurement record;
- molecular coordinates together with relevant momenta or collective variables;
- robot joint angles, angular velocities, and sensor readings.

A poor state representation can make an otherwise Markovian process appear history-dependent.

Partial observability

The true physical state may not be directly available to the agent.

- A single quantum measurement does not reveal a complete wavefunction or density matrix.
- A robot has noisy sensors and cannot observe every environmental variable.
- Molecular simulations may use only a few collective variables from a high-dimensional configuration.

The agent then acts from an **observation** or a summary of past observations rather than from the complete state.

For Week 1: we use the Markov framework as the basic model, while remembering that state construction is part of the problem.

Policy and environment play complementary roles

The policy gives

$$\mathbb{P}(A_t = a_t \mid S_t = s_t) = \pi(a_t \mid s_t),$$

while the environment gives

$$\mathbb{P}(S_{t+1} = s_{t+1}, R_{t+1} = r_{t+1} \mid S_t = s_t, A_t = a_t) = p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

Changing the policy changes which actions are selected, which changes the distribution over future trajectories even when the environment kernel remains fixed.

Probability of a trajectory

Let the initial state be drawn from

$$\mu_0 \in \text{Prob}(S), \quad \mu_0(s) = \mathbb{P}(S_0 = s).$$

For a finite trajectory

$$\tau = (s_0, a_0, r_1, s_1, \dots, a_{T-1}, r_T, s_T),$$

the Markov property and chain rule give

$$P_\pi(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

Two sources of randomness

The trajectory probability separates two sources of stochasticity:

$$P_{\pi}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \underbrace{\pi(a_t \mid s_t)}_{\text{agent}} \underbrace{p(s_{t+1}, r_{t+1} \mid s_t, a_t)}_{\text{environment}}.$$

- The policy may randomize its action choices.
- The environment may respond stochastically even to a fixed state and action.

These are conceptually distinct and may both be present in a physical problem.

Immediate reward is not enough

Suppose a robot must step away from a target to avoid an obstacle, or a quantum controller must temporarily reduce fidelity before reaching a better final state.

The immediate reward R_{t+1} cannot by itself determine whether A_t was a good action.

We therefore evaluate actions through the rewards that follow them along the trajectory.

Takeaway. RL is concerned with accumulated future reward rather than only the next reward.

The discounted return

For a continuing task, the return from time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

For an episode terminating at time T ,

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

The return assigns one number to the future consequences of the current point in a trajectory.

The discount factor

The parameter

$$\gamma \in [0, 1]$$

controls the relative weight of future rewards.

- $\gamma = 0$: only the next reward matters;
- small γ : short-term consequences dominate;
- γ close to 1: rewards far in the future remain important.

Discounting can encode preference for earlier reward and can also keep infinite-horizon returns mathematically well behaved.

The recursive identity for returns

The return satisfies

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Indeed,

$$\gamma G_{t+1} = \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots,$$

so adding R_{t+1} reconstructs G_t .

Takeaway. This simple recursion is the source of the Bellman equations introduced later in the course.

The policy objective

If the initial state is sampled from μ_0 , define

$$J_{\mu_0}(\pi) = \mathbb{E}_{\pi}[G_0].$$

Equivalently,

$$J_{\mu_0}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

The reinforcement-learning objective is

$$\pi^* \in \arg \max_{\pi} J_{\mu_0}(\pi).$$

The expectation is taken over trajectories generated jointly by μ_0 , π , and p .

State-value functions

The objective $J_{\mu_0}(\pi)$ gives one number for an entire policy. To reason locally, we ask what return to expect from a particular state. For a fixed policy π ,

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

Interpretation:

$v_{\pi}(s)$ is the expected future return when the process starts from state s and all future actions are chosen according to π .

Action-value functions

The action-value function additionally conditions on the current action:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Interpretation:

$q_{\pi}(s, a)$ is the expected return obtained by first taking action a in state s , and then following policy π .

It allows the long-term consequences of different admissible actions to be compared at the same state.

Relation between v_π and q_π

A stochastic policy chooses action a with probability $\pi(a \mid s)$.
Therefore,

$$v_\pi(s) = \sum_{a \in A(s)} \pi(a \mid s) q_\pi(s, a).$$

The value of a state is the average action value under the policy's action distribution.

For a deterministic policy μ ,

$$v_\mu(s) = q_\mu(s, \mu(s)).$$

Why value functions matter

Rewards are attached to individual transitions, while value functions summarize long-term consequences.

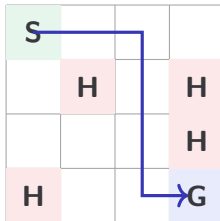
They allow us to ask:

- Which states are promising under the current policy?
- Which action has the best long-term outcome in a given state?
- How does a locally different action alter expected future reward?
- How can a policy be evaluated or improved?

Value functions turn a trajectory-level objective into quantities associated with states and state–action pairs.

Notebook example: a small grid world

Consider a 4×4 grid. The agent starts at S , aims for G , and should avoid holes H .



- State: grid position
- Actions: left, down, right, up
- Reward: goal reward, hole penalty, small step cost
- Episode: ends at a goal, hole, or maximum number of steps

What an environment does

A minimal environment exposes two central operations:

```
state = env.reset()

next_state, reward, terminated, truncated = env.step(action)
```

`reset()`: starts a new episode and returns the initial state;

`step(action)`: applies an action and returns the next state,
reward, and termination information.

The notebook first implements this mechanism directly, before using an external RL library.

Gymnasium and FrozenLake

Gymnasium provides standard environments with the same interface. For FrozenLake-v1:

- states encode positions on a grid;
- actions are integers representing left, down, right, and up;
- the environment can be deterministic or “slippery”;
- the goal is to reach the terminal goal state without falling into a hole.

Week 1 lesson: the hand-written grid world and FrozenLake have the same mathematical structure. Gymnasium supplies the mechanics, not the learning algorithm.

Three application pathways

We now reuse the same RL vocabulary in three domains:

Domain	Main challenge	Illustrative task
Quantum technology	uncertainty and partial information	prepare a target qubit state
Molecular dynamics	large configuration spaces and rare events	cross a double-well barrier
Robotics/control	real-time dynamics and disturbances	stabilize an upright body

Quantum technology: dealing with uncertainty

Quantum systems are natural sequential decision problems because:

- measurement outcomes are probabilistic;
- measurements may disturb the state;
- the complete state is rarely available from a single observation;
- control decisions may depend on previous measurement outcomes;
- noise and calibration errors make an exact model imperfect.

RL can learn a measurement-conditioned or state-estimate-conditioned control policy from repeated interaction.

Quantum technology as an RL problem

State: wavefunction, density matrix, Bloch vector, measurement record, or estimated parameters;

Action: pulse, field amplitude, gate, measurement basis, or feedback operation;

Reward: fidelity, low energy, information gain, successful correction, or robustness;

Episode: one control sequence, experiment, circuit execution, or preparation attempt.

Applications include state preparation, gate optimization, adaptive sensing, and quantum error correction.

Toy example: qubit state preparation

A real pure qubit state can be parametrized by

$$|\psi(\theta)\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + \sin\left(\frac{\theta}{2}\right) |1\rangle.$$

The initial state is $\theta = 0$, corresponding to $|0\rangle$, and the target is $\theta = \pi$, corresponding to $|1\rangle$.

- State: current angle θ
- Action: choose a small or large rotation pulse
- Episode: at most eight control steps

Quantum reward design and policy tradeoffs

The target fidelity is

$$F(\theta) = \cos^2\left(\frac{\theta - \theta_{\text{target}}}{2}\right).$$

A simple step reward is

$$R_{t+1} = F(\theta_{t+1}) - F(\theta_t) - \text{control cost}.$$

Three hand-written strategies illustrate the tradeoff:

- cautious: small pulses are cheap but slow;
- aggressive: large pulses are fast but costly;
- mixed: large pulses far from the target, small pulses near it.

A learning agent would try to discover such a state-dependent strategy.

Molecular dynamics: dealing with complexity

Molecular systems evolve in very large configuration spaces.
Important processes can be rare on ordinary simulation timescales:

- conformational changes and protein folding;
- ligand binding and unbinding;
- chemical reactions;
- transitions between metastable states.

RL can choose interventions or sampling decisions that help trajectories explore dynamically important regions more efficiently.

Molecular dynamics as an RL problem

State: coordinates, velocities, distances, angles, or collective variables;

Action: biasing force, external field, temperature schedule, starting configuration, or next sampling region;

Reward: reach a rare state, discover a transition, reduce uncertainty, or improve sampling efficiency;

Episode: one trajectory, transition attempt, or adaptive sampling round.

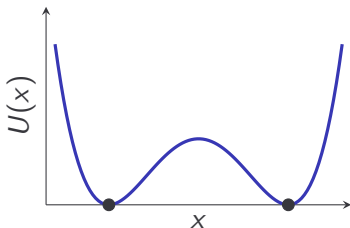
The central difficulty is deciding where limited simulation effort is most informative or useful.

Toy example: crossing a double-well barrier

Consider the potential

$$U(x) = (x^2 - 1)^2,$$

with stable regions near $x = -1$ and $x = 1$.



The particle begins near $x = -1$ and should reach the right well.

Actions apply a force left, right, or no force.

Noisy molecular dynamics and reward

A simplified update is

$$x_{t+1} = x_t + \Delta t [F_{\text{physical}}(x_t) + F_{\text{control}}] + \text{noise},$$

where

$$F_{\text{physical}}(x) = -\frac{dU}{dx} = -4x(x^2 - 1).$$

A reward can combine progress and control cost:

$$R_{t+1} = |x_{\text{target}} - x_t| - |x_{\text{target}} - x_{t+1}| - c|F_{\text{control}}|,$$

with an additional bonus for reaching the target well.

Robotics and control: dealing with real-time dynamics

A robot must continuously react to a physical system that evolves while decisions are being made.

Challenges include:

- nonlinear and coupled dynamics;
- disturbances, friction, and uncertain loads;
- actuator limits and safety constraints;
- delayed consequences of motor commands;
- the need for rapid feedback decisions.

RL is useful when an analytical point-to-point solution is unavailable or insufficiently robust.

Robotics and control as an RL problem

State: sensor readings, positions, joint angles, velocities, camera observations, or maps;

Action: motor torques, joint commands, wheel speeds, or high-level navigation commands;

Reward: task success, stability, smoothness, energy efficiency, and collision avoidance;

Episode: one attempt at balancing, walking, grasping, or navigation.

Applications include locomotion, manipulation, navigation, motor control, and sim-to-real transfer.

Toy example: balancing under random disturbances

Model the body as an unstable inverted pendulum with angle θ and angular velocity ω .

A simple discrete update is

$$\omega_{t+1} = \omega_t + \Delta t [g \sin \theta_t - d\omega_t + \tau(a_t) + w_t],$$

$$\theta_{t+1} = \theta_t + \Delta t \omega_{t+1}.$$

The action $a_t \in \{-1, 0, 1\}$ moves the arms left, keeps them centered, or moves them right. The disturbance w_t models wind.

A control reward

A simple reward is

$$R_{t+1} = -\theta_{t+1}^2 - 0.05 \omega_{t+1}^2 - 0.02 a_t^2.$$

This rewards the robot for

- staying close to upright;
- avoiding large angular velocity;
- using moderate control effort.

A hand-written stabilizing rule can illustrate the task. Later, an RL algorithm can learn the feedback rule from many balancing episodes.

The same framework across three domains

	Quantum control	Molecular dynamics	Robotics/control
State	state estimate or measurement record	coordinates or collective variables	angle, velocity, sensors
Action	pulse or measurement	biasing force or sampling choice	torque or motor command
Reward	fidelity or information	progress, transition, efficiency	stability and task success
Main issue	uncertainty	complexity and rare events	real-time dynamics

The physical interpretation changes; the mathematical skeleton remains the same.

Preview: is the environment model known?

The transition kernel $p(s', r \mid s, a)$ may be:

- known from equations of motion or a trusted simulator;
- approximately known but imperfect;
- learned from data;
- unavailable and accessible only through interaction.

This distinction motivates two broad approaches:

Model-based or model-aware: use a model to evaluate or plan future transitions;

Model-free: estimate values or policies directly from sampled experience.

Preview of the next mathematical step

The return recursion

$$G_t = R_{t+1} + \gamma G_{t+1}$$

leads to self-consistency relations for value functions: the value now is connected to immediate reward and value later.

Later in the course we will study:

- Bellman equations and dynamic programming when the model is known;
- Monte Carlo estimation from complete sampled episodes;
- temporal-difference methods that update from partial trajectories;
- policy-gradient and actor–critic methods.

Today, the important foundation is the common probabilistic framework.

Summary and discussion questions

Core ideas

- RL differs from fixed-data learning because actions influence future data.
- The policy π and environment kernel p jointly generate trajectories.
- The Markov state should summarize information relevant to the one-step future.
- Returns and value functions describe long-term consequences.
- The same formalism applies to uncertainty, complexity, and real-time dynamics.

Discuss

1. What is the state, action, and reward in a scientific problem you care about?
2. Is its state fully observable and Markovian?
3. Which part of the environment model is known from physics?